



Week 2

Sadia 'Rain' Sharmin

Classes begin 10 minutes after the hour

THE FIRST CODER?



This Week's Cool Programmer(s) Feature: *Ada Lovelace*



Linked Lists

It's all about the links

-  Traversals
-  Insertion
-  Deletion
-  Efficiency

Abstract Data Types

An abstract data type is not defined in code

The definition of an abstract data type includes:

- The data it stores
- The operations that can be performed

Abstract data types are “a common language” among computer scientists

e.g., set, list, map, multiset, iterable

Abstract Data Types

List ADT:

Data: an ordered sequence of elements
(possibly with duplicates)

Operations: get size, access element by
index, insert a value at a given index,
remove a value at a given index

Python list

Python lists are **array-based lists**

Data is stored in a **contiguous block of memory**

That is, list elements are stored in consecutive blocks of memory, one after another (i.e., no gaps)

Address	Memory	
⋮	⋮	
400	601	(list index 0)
401	512	
402	699	
403	650	
404		
⋮	⋮	
512	7	
⋮	⋮	
601	8	
⋮	⋮	
650	-3	
⋮	⋮	
699	100	
⋮	⋮	

List operations

Due to the way list data is stored:

Operation	Running time ($n = \text{len}(\text{lst})$)
List indexing (<code>lst[i]</code>)	$\Theta(1)$
List index assignment (<code>lst[i] = ...</code>)	$\Theta(1)$
List insertion at end (<code>list.append(lst, ...)</code>)	$\Theta(1)$
List deletion at end (<code>list.pop(lst)</code>)	$\Theta(1)$
List insertion at index (<code>list.insert(lst, i, ...)</code>)	$\Theta(n - i)$
List deletion at index (<code>list.pop(lst, i)</code>)	$\Theta(n - i)$

Alternative list implementation

Array-based lists store references to elements in contiguous blocks of memory.

Linked lists can store elements anywhere, but each element must store a reference to the next element in the list.

Our goals this week

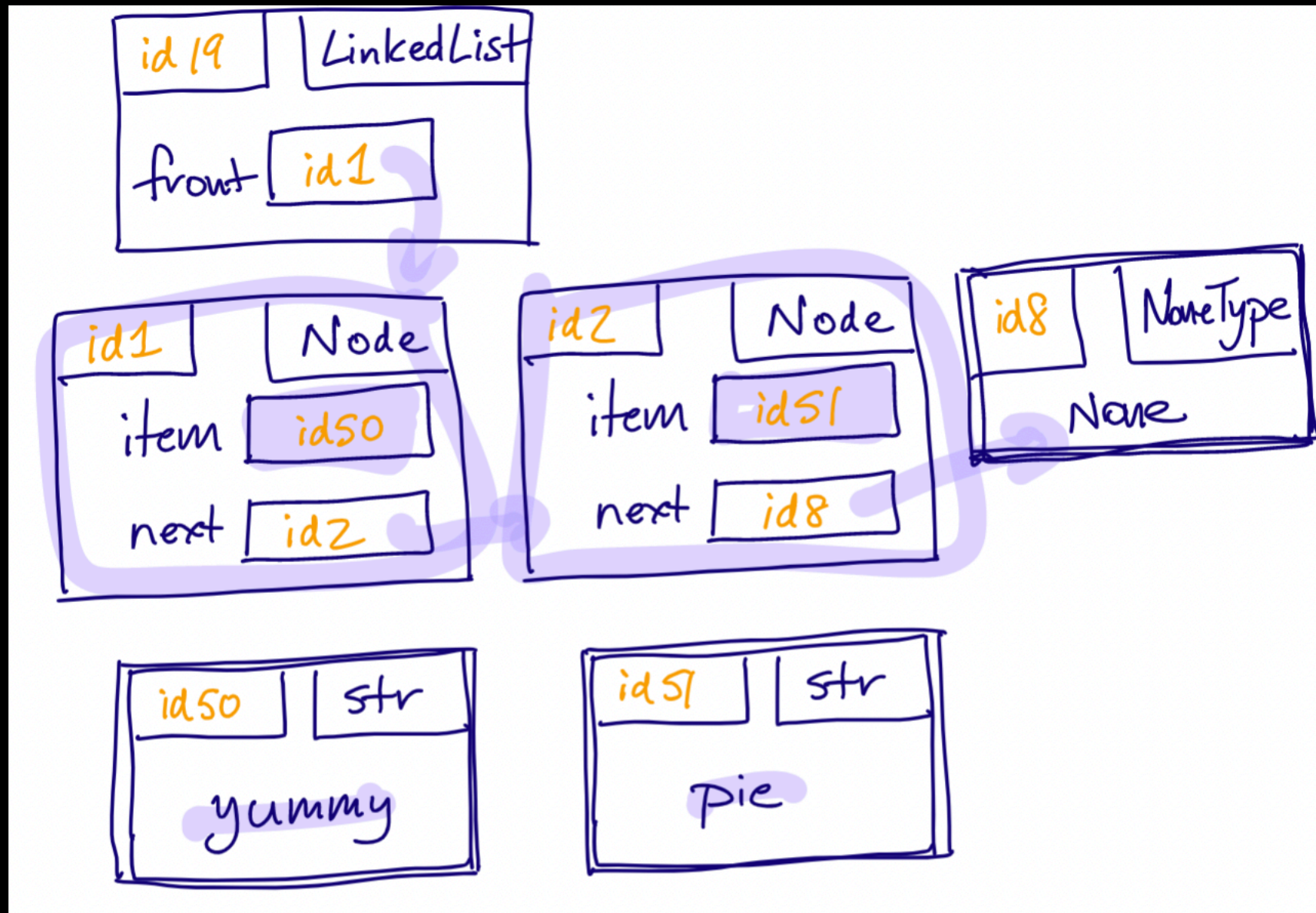
We're going to develop a new class `LinkedList` that has the same public interface as `list` but is implemented using nodes instead of arrays.

We'll see that:

- linked lists are faster than array-based lists for some operations, but not others
- (later) the concept of nodes can be applied to create more complex data structures, not just linked lists

Linked List Traversals

Visualizing Linked Lists

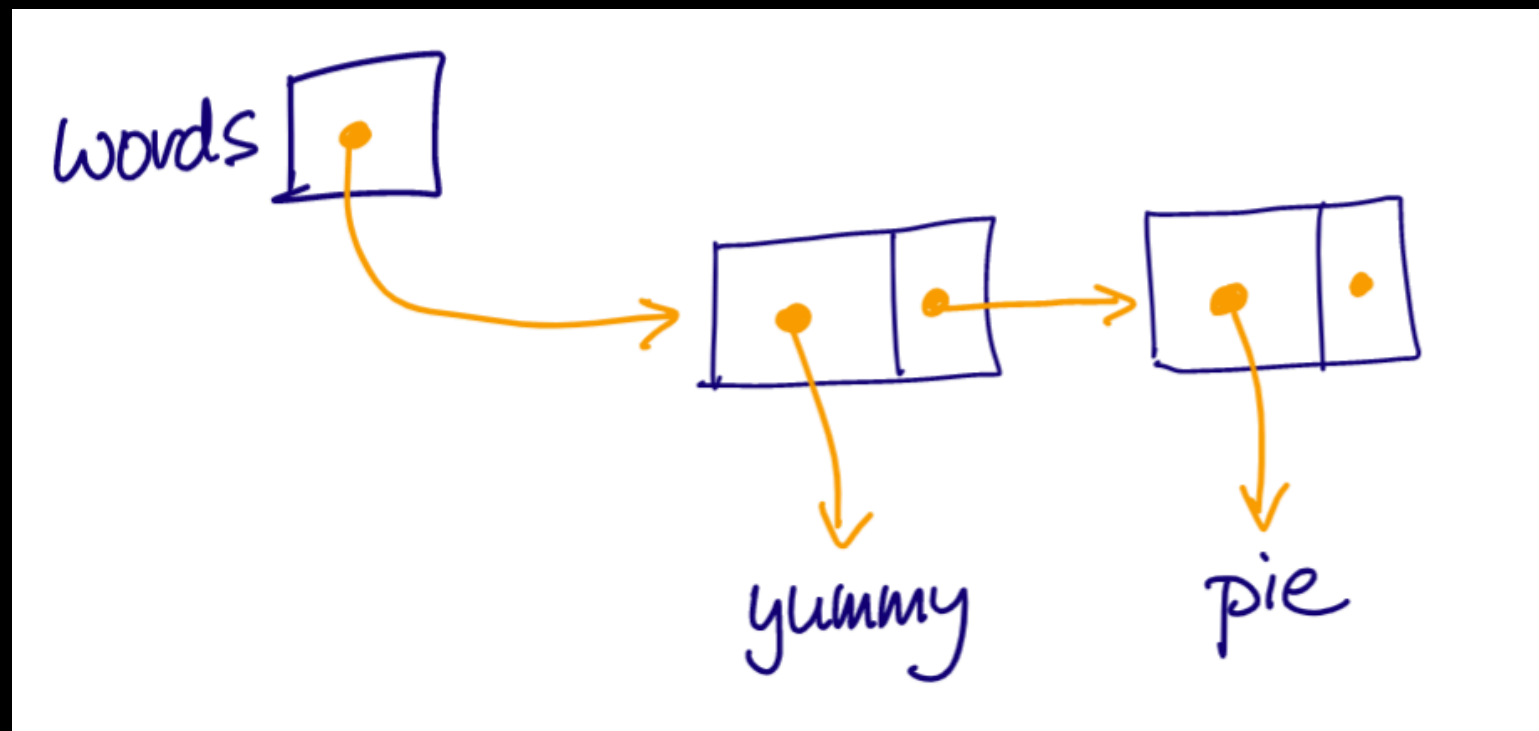


Here is a memory model diagram of a LinkedList with two elements.

We are going to switch to a more abstract shorthand for this.

Visualizing Linked Lists

Here is a more abstract shorthand for the same linked list represented in the memory model diagram we just saw:

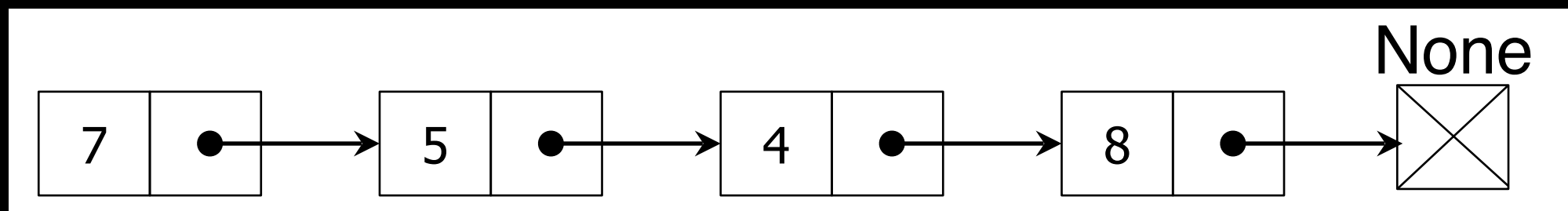


Visualizing Linked Lists



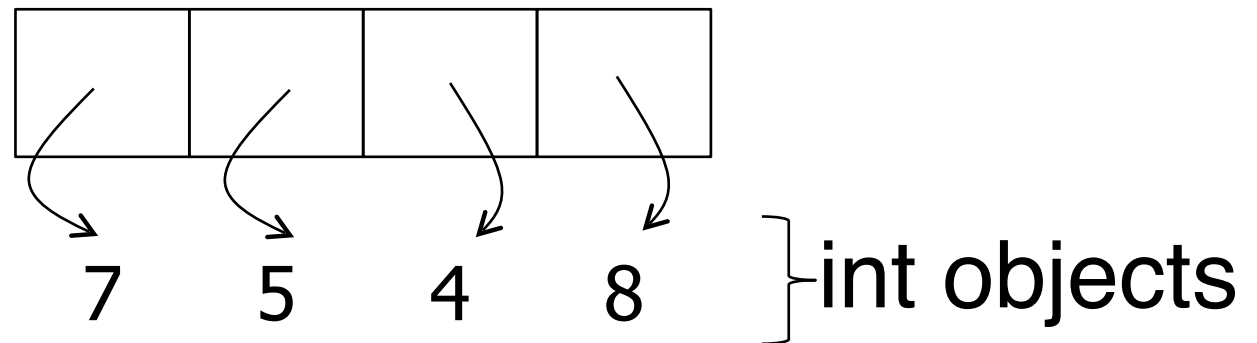
It's all about the links; that's what we focus on today.

Visualizing Linked Lists

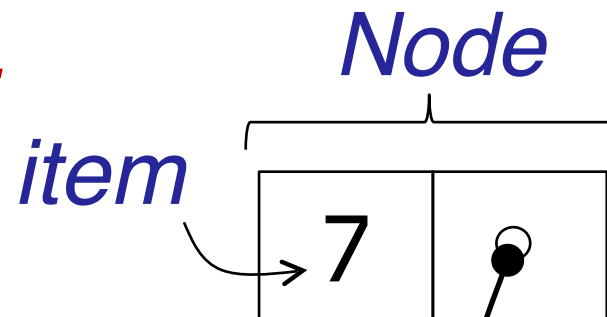


Drawing Linked Lists

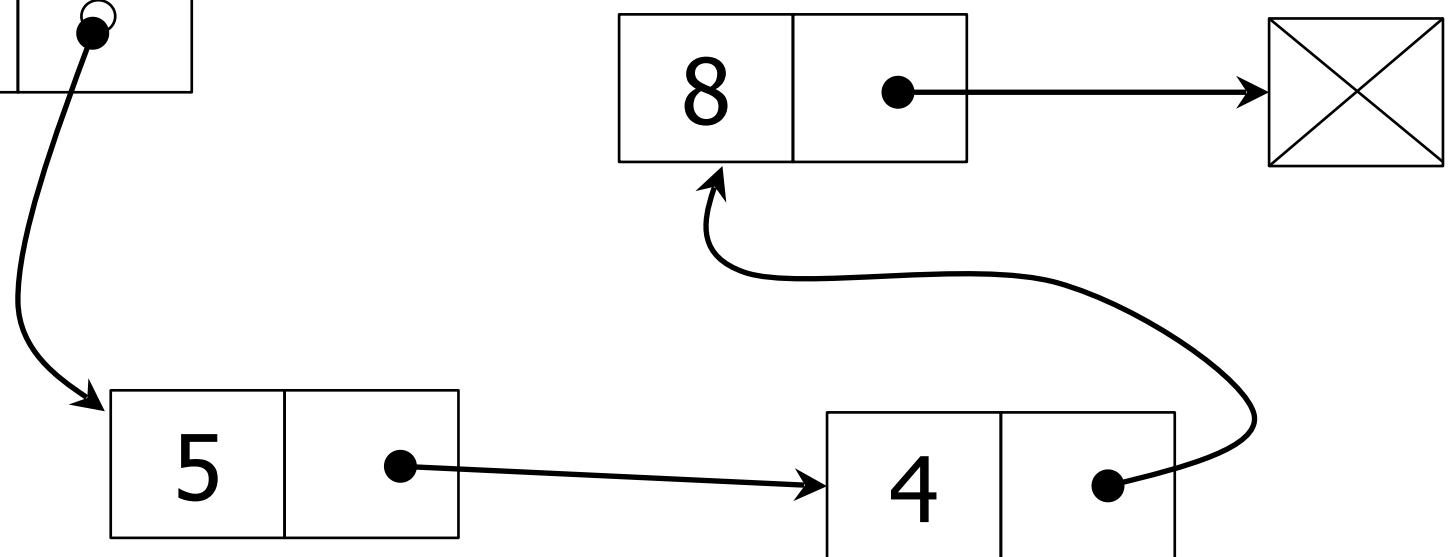
Python list: [7, 5, 4, 8]



Linked list:



reference to next Node

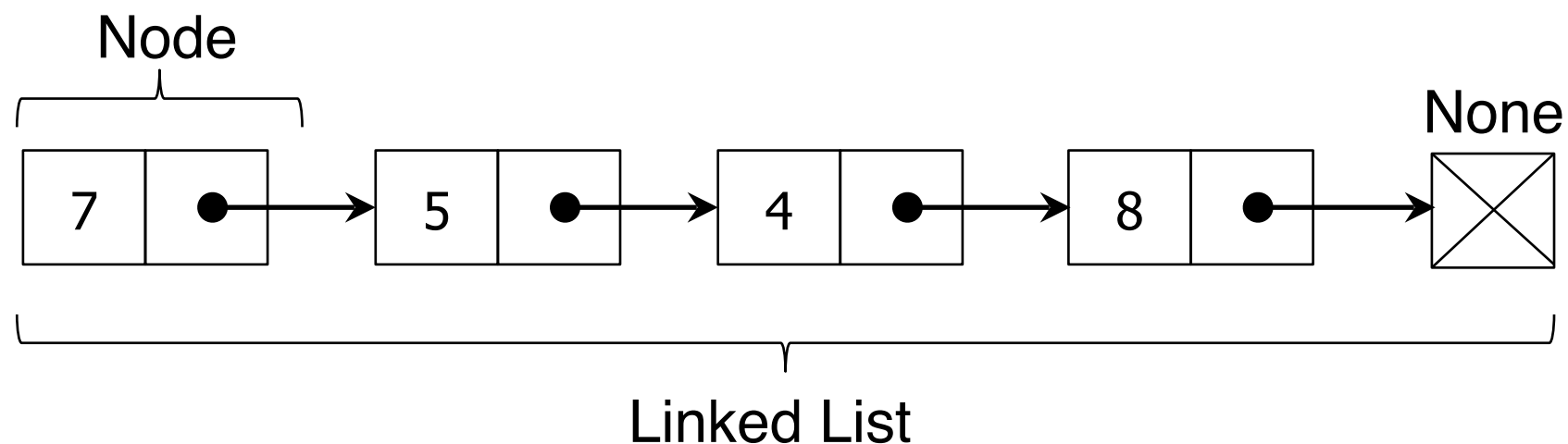


Get in the habit of drawing diagrams to visualize things better!

Thinking about linked lists

There are two useful, but different, ways of thinking of linked list nodes:

1. As a list made up of an item (value) and a sub-list (rest)
2. As objects (nodes), each containing a value and a reference to another similar node object (the “next link in the chain”)





WORKSHEET

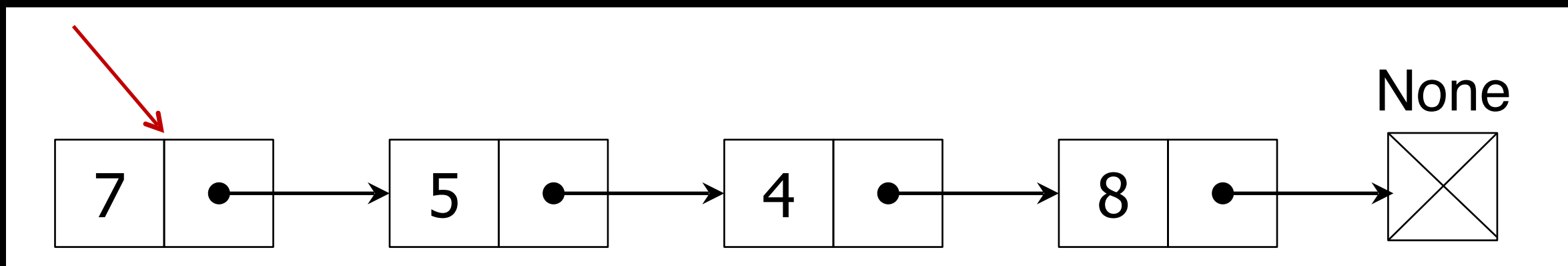
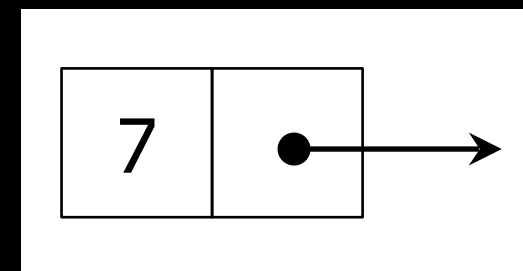
Introduction to Linked Lists

Code summary

We have a class for a linked list node, then a separate "wrapper" to represent the linked list as a whole. These are our data structures:

```
class _Node:  
    item: Any  
    next: Optional[_Node] = None
```

```
class LinkedList:  
    _first: Optional[_Node]
```

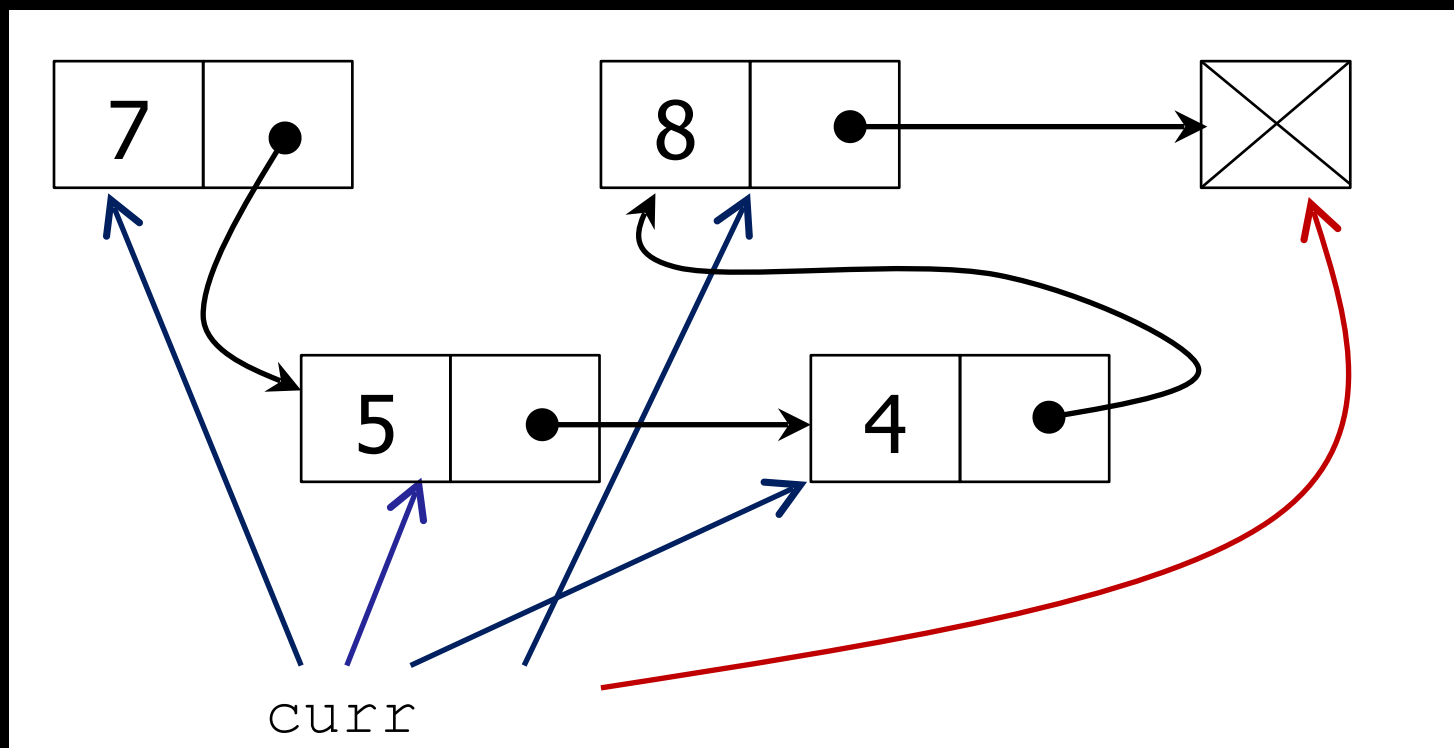


Walking a list

Make a reference to (at least one) node, and move it along the list (a very common pattern):

```
curr = self._first
while <some condition here...>:
    # do something here ...
    curr = curr.next
```

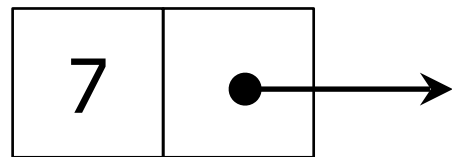
What if curr is None?



Linked lists code summary

Data structures:

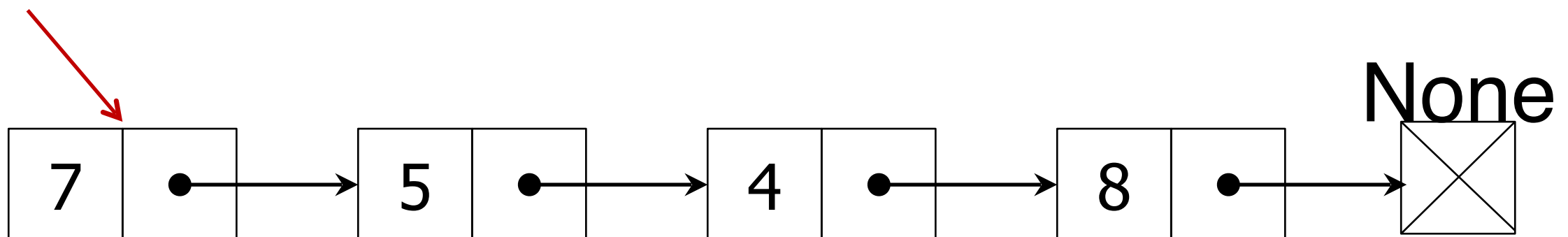
```
class _Node:
    item: Any
    next: Optional[_Node]
```



```
class LinkedList:
    _first: Optional[_Node]
```

Traversal idea:

```
curr = self._first
while curr is not None:
    ... curr.item ...
    curr = curr.next
```





WORKSHEET

Traversing Linked Lists

Linked Lists

Insertion

Insert

We might want to implement all sorts of insert variations depending on what operations we want the linked list to support, e.g.

- Prepend

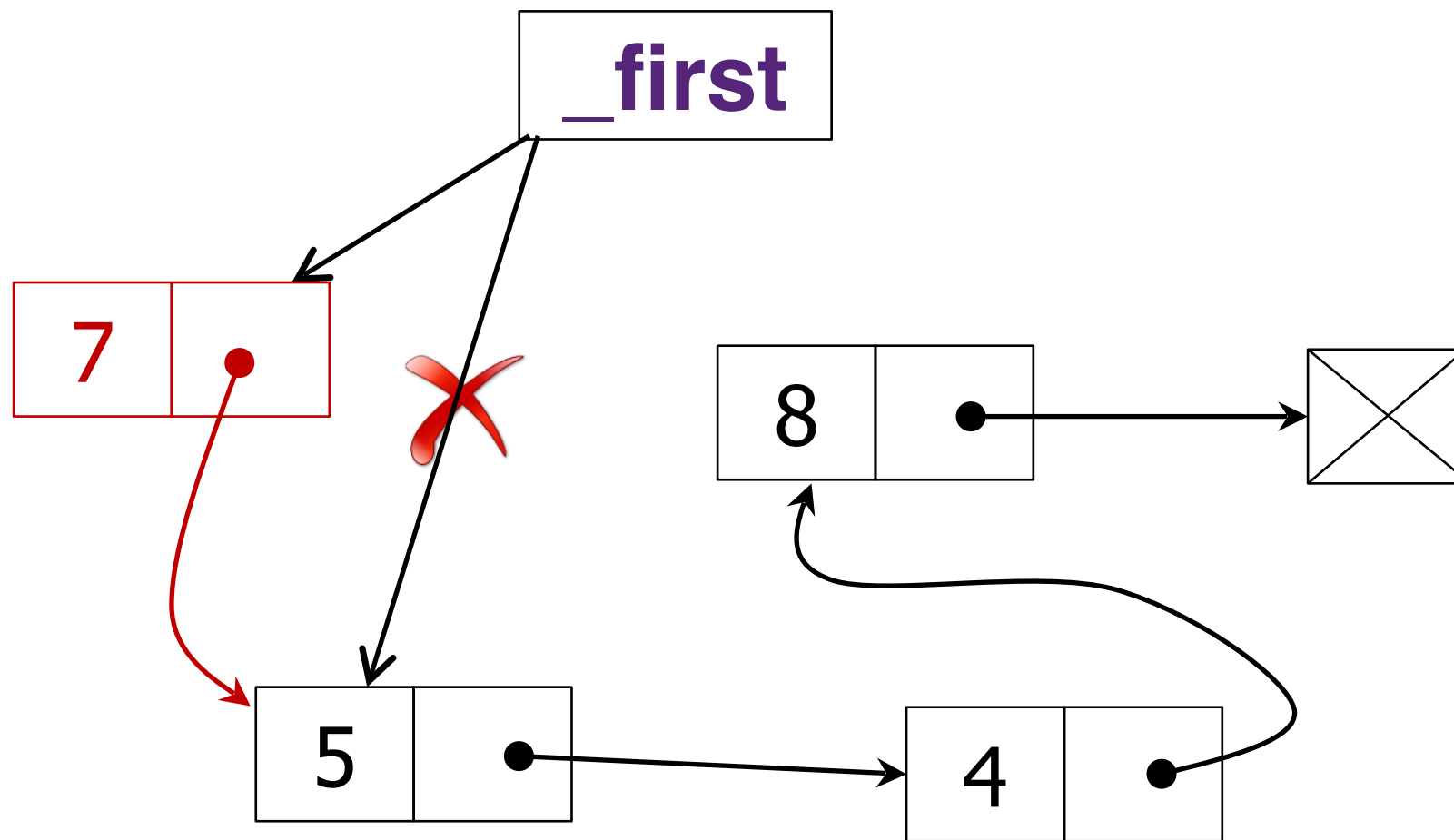
- Append

- Insert at a given index

Use diagrams to visualize such operations!

prepend (insert at the front)

Easy: simply adjust the `_first` reference



Insertion

append

We'll need to change...

some node inside the list

possibly the last node

possibly `_first` .. why?

Always draw diagrams!

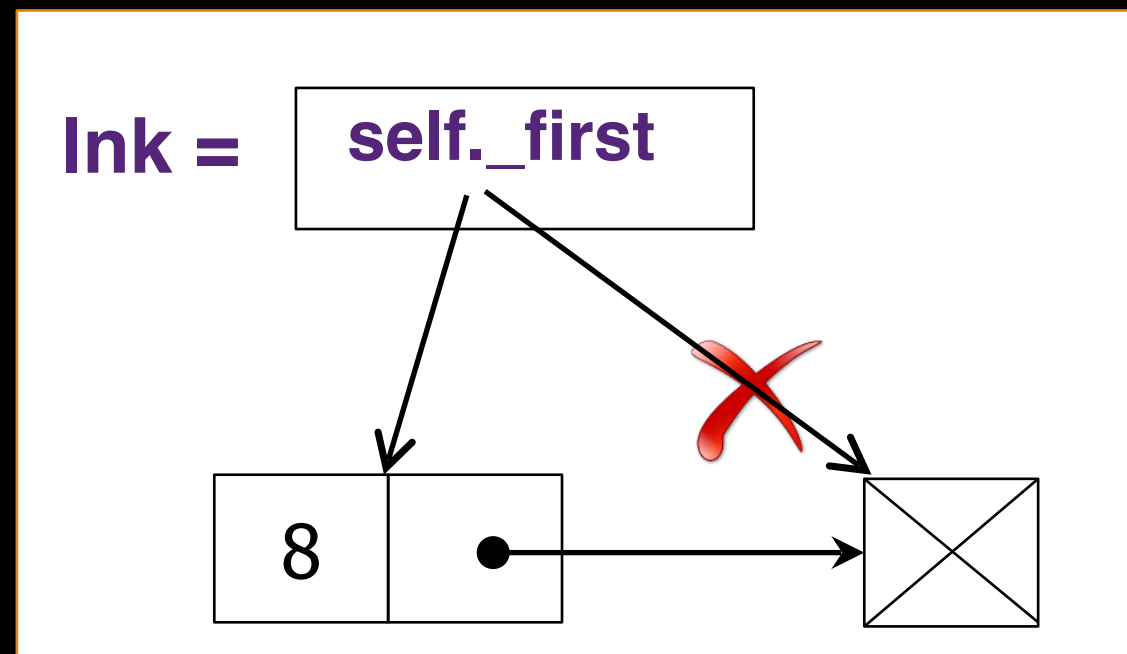
append to empty list

First node being appended

Sort of similar to prepend in this particular corner case...

List is initially
empty.

Appending a
new node.



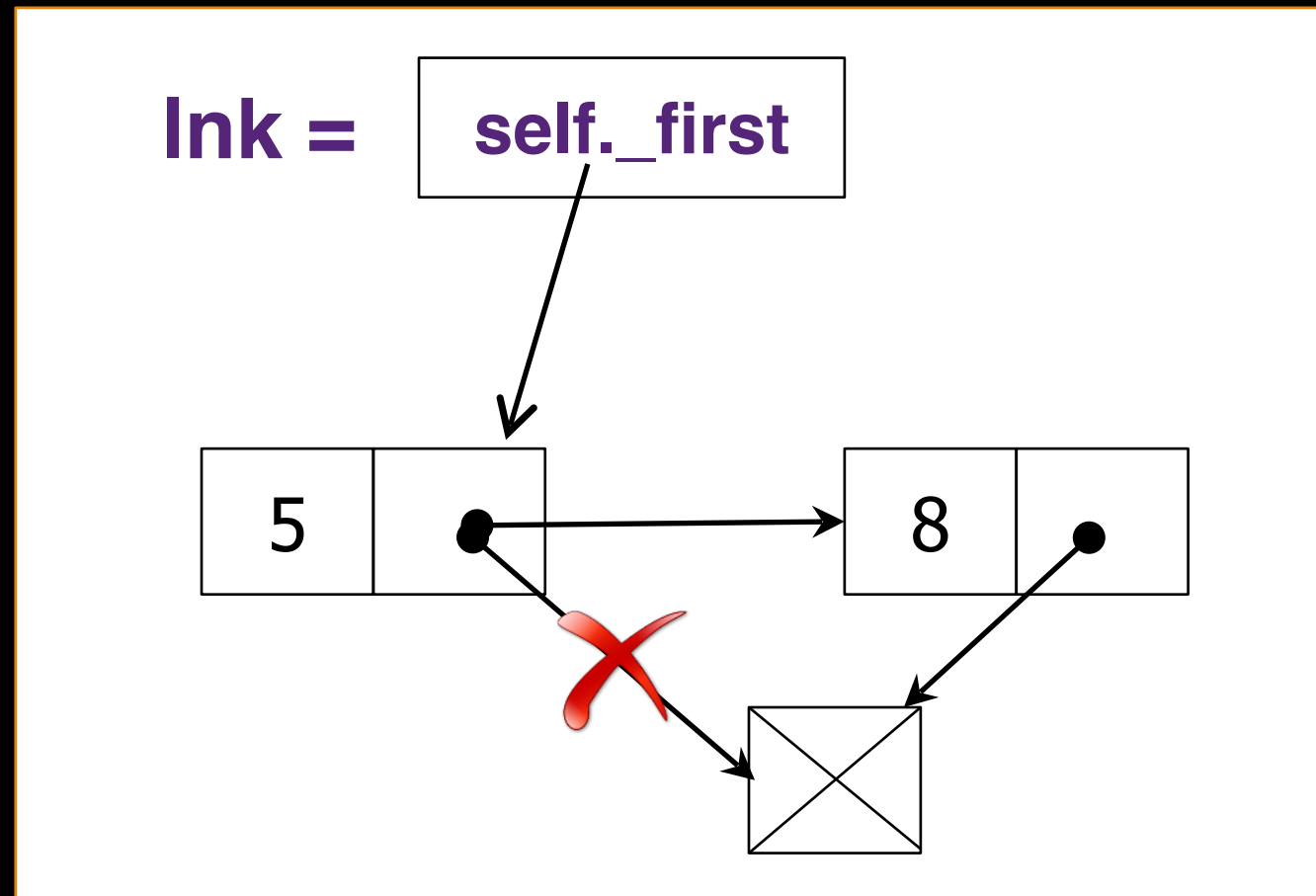
Always draw diagrams!

append to 1 item list

Second node being appended

List has one
element (5).

Appending a
new node (8).

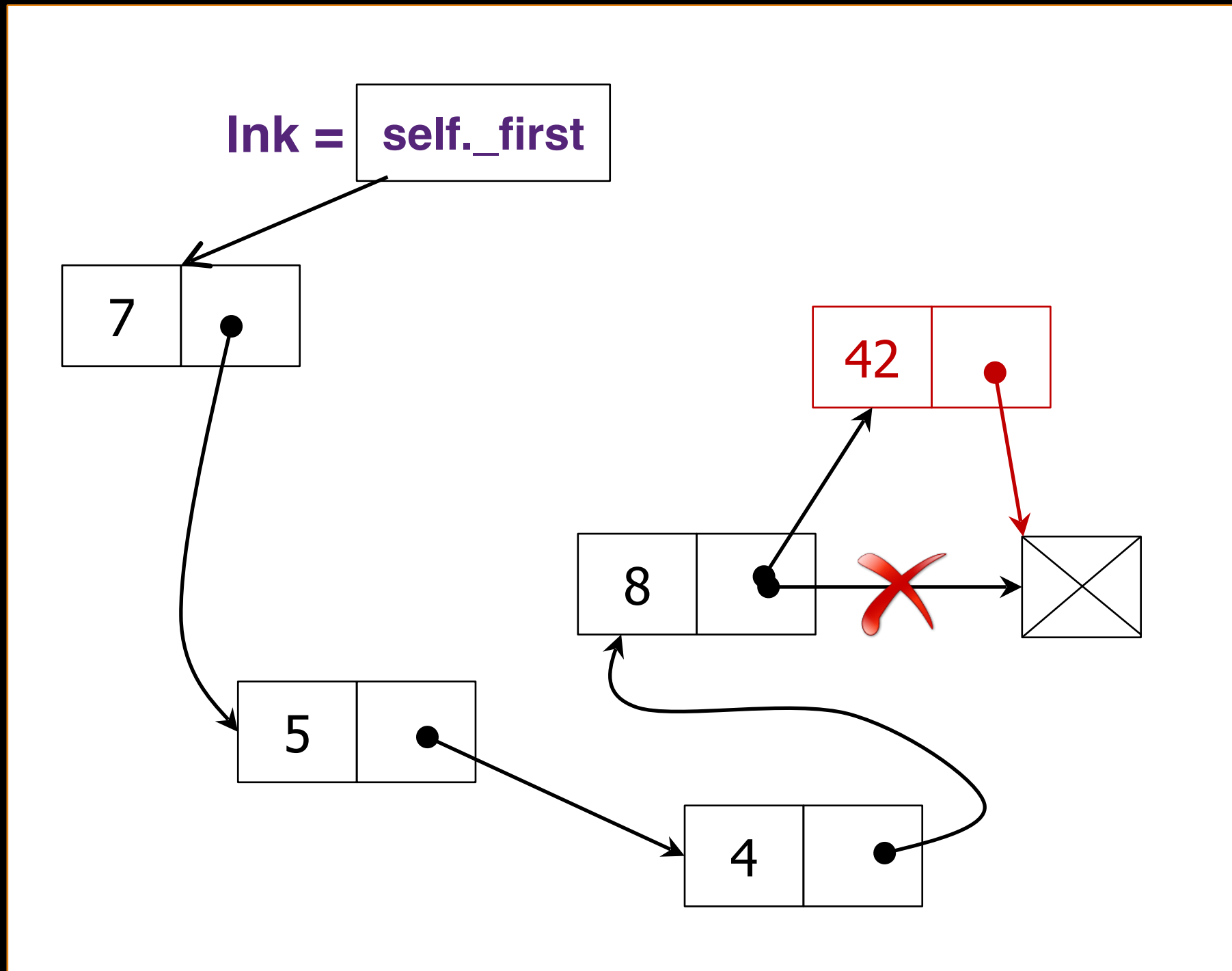


Always draw diagrams!

Insertion

append to many item list

Insertion



Always draw diagrams!

Insertion

Insert at given index

Now let's work on writing the function to insert at a given index.



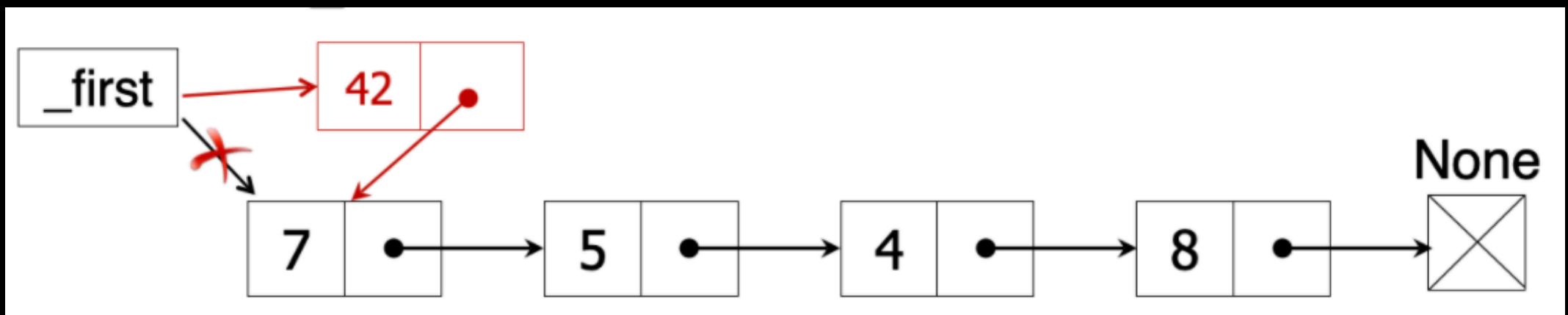
WORKSHEET

Mutating Linked Lists

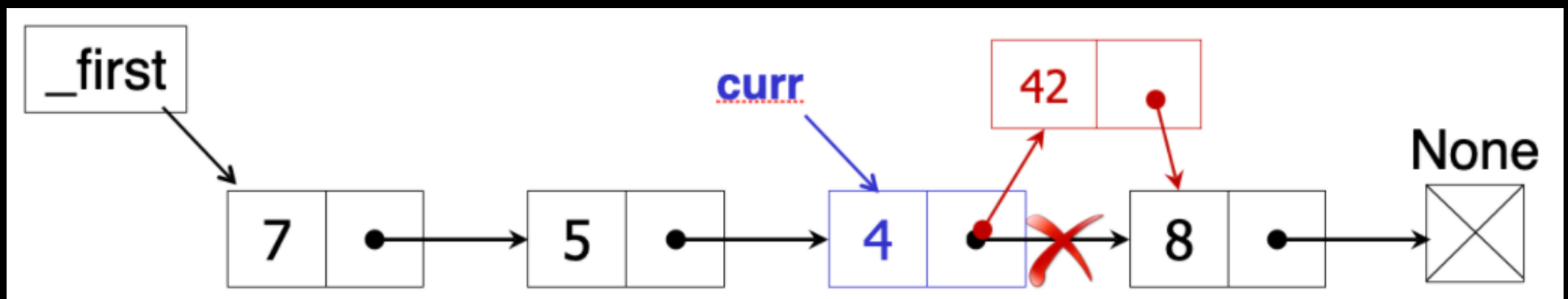
Exercise 1: Index-based insertion

Recapping the key ideas

1. Figure out when we need to modify `self._first` vs. a `_Node` in the list.



2. When `index > 0`, iterate to the $(\text{index}-1)^{\text{th}}$ node and update links.



Linked Lists

Deletion

Delete / pop

We might want to implement all sorts of delete variations depending on what operations we want the linked list to support, e.g.

- Delete from the front

- Delete from the back

- Delete (pop) from any index in the list

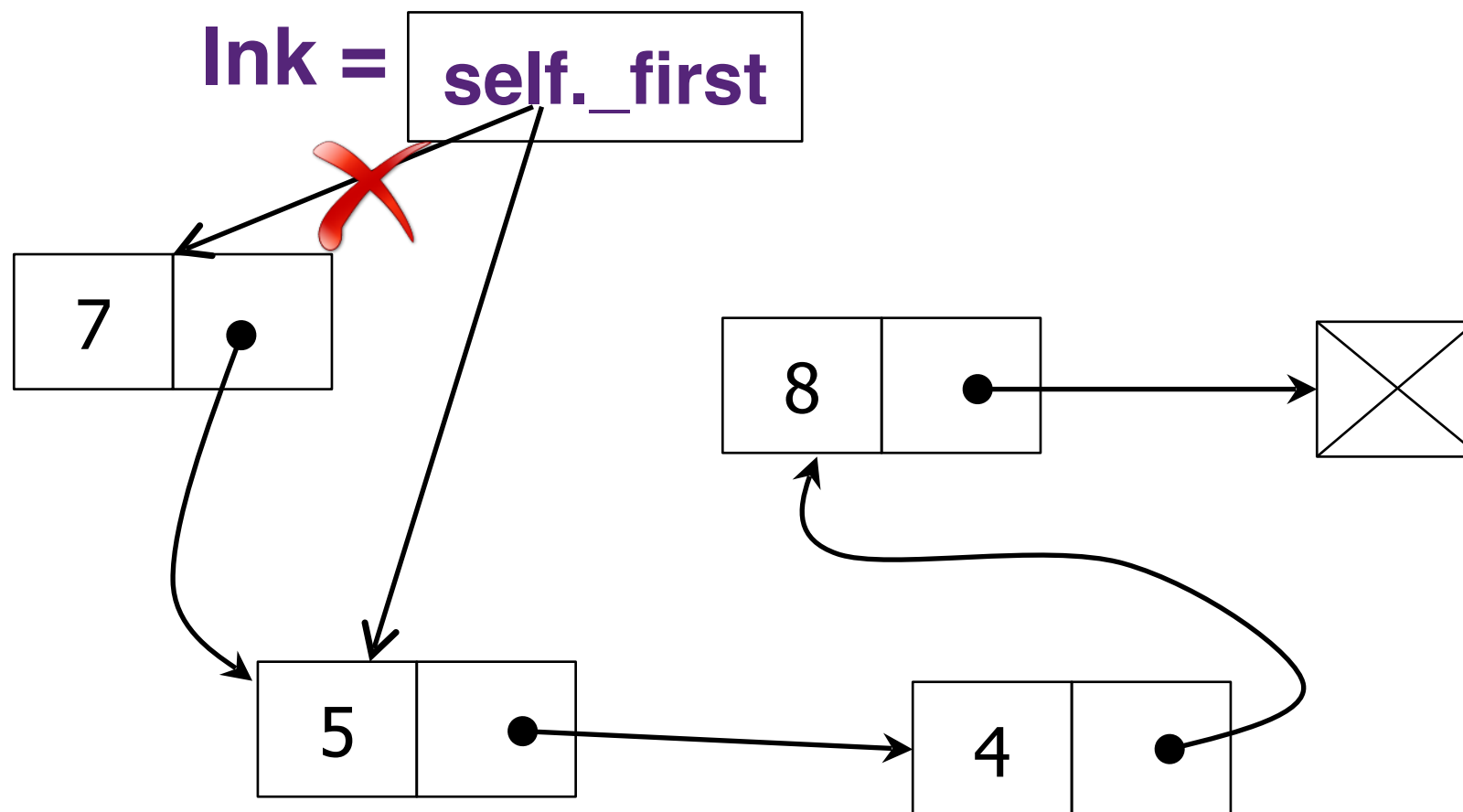
Ultimately, the "pop from an index" version covers all cases

- But, let's visualize such operations first ...

Delete from front

Easy: make `_first` reference the second node (garbage collection takes care of former first node automatically)

No need to walk the list ...



Delete from front

Easy: make `_first` reference the second node
(garbage collection takes care of former first node automatically)

Consider corner cases though:

What if only one node?

What if list is empty?

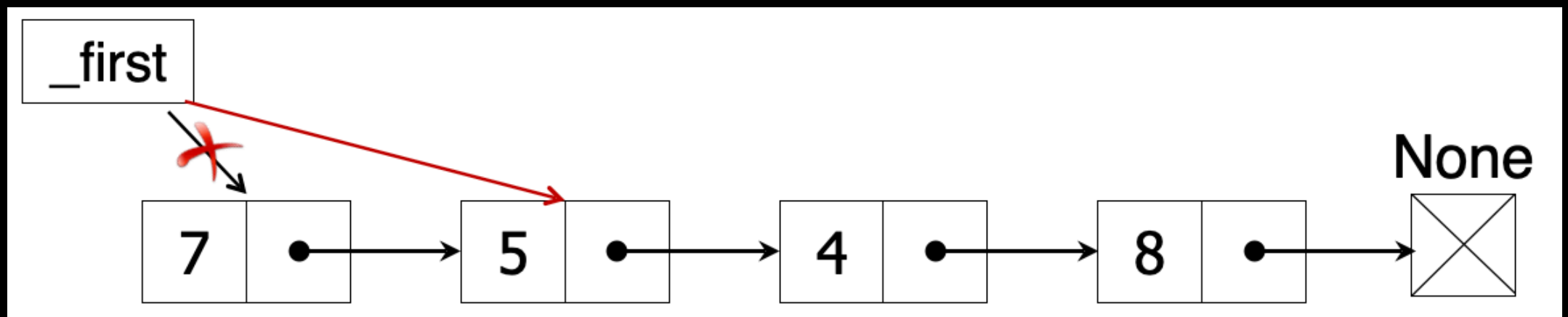


WORKSHEET

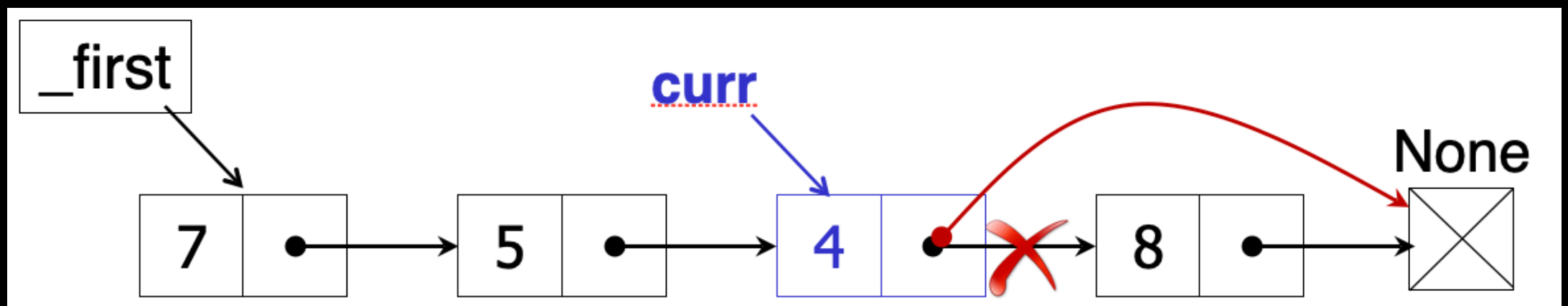
Exercise 2: Index-based Deletion

Same key ideas!

1. Figure out when we need to modify `self._first` vs. a `_Node` in the list.



2. When `index > 0`, iterate to the $(\text{index}-1)^{\text{th}}$ node and update links.



The “problem of previous”

Strategy #1: iterate to the node *before* the desired position.

```
i = 0
curr = self._first
while not (curr is None or i == index - 1):
    curr = curr.next
    i += 1
```

The “problem of previous”

Strategy #2: track the previous node explicitly

```
i = 0
prev = None
curr = self._first
while not (curr is None or i == index):
    prev, curr = curr, curr.next
    i += 1
```



WORKSHEET

Exercise 3: Value-based Deletion

Linked Lists

Efficiency

Recall that...

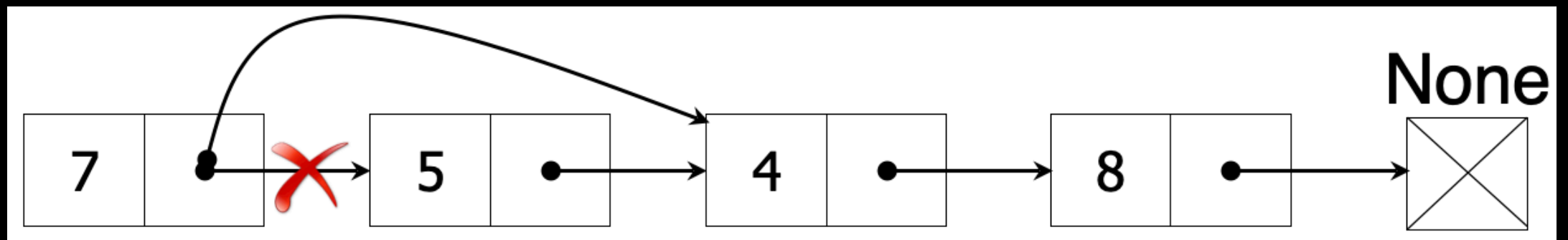
Python's lists are array-based. Each list stores the ids of its elements in a contiguous block of memory.

Every insertion and deletion causes every element after the changed index to move.

Operation (assuming $0 \leq i < n$)	Running time
Indexing (<code>lst[i]</code>)	$\Theta(1)$
Insert into index i	$\Theta(n - i)$
Remove item at index i	$\Theta(n - i)$

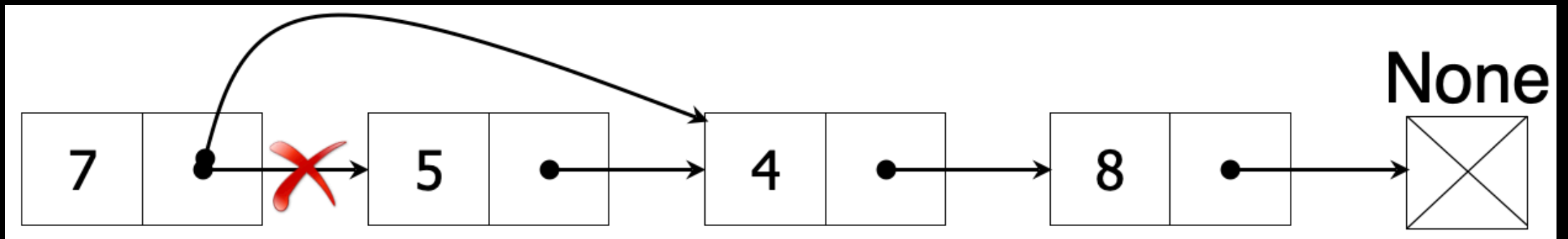
What about linked lists?

Remove an element from beginning – much faster (no need to "shift" anything)

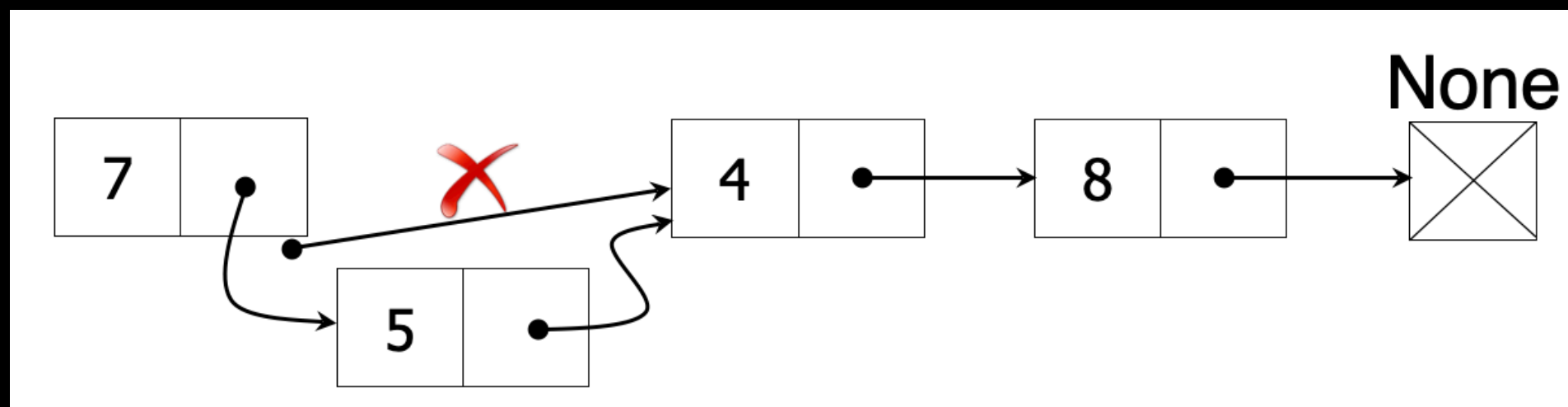


What about linked lists?

Remove an element from beginning – much faster (no need to "shift" anything)



Insert an element near the beginning – no need to shift subsequent elements



What about linked lists?

Making changes to the beginning of a linked list:

Just adjust a couple of references, no moving memory!

Efficiency

Compare and contrast ...

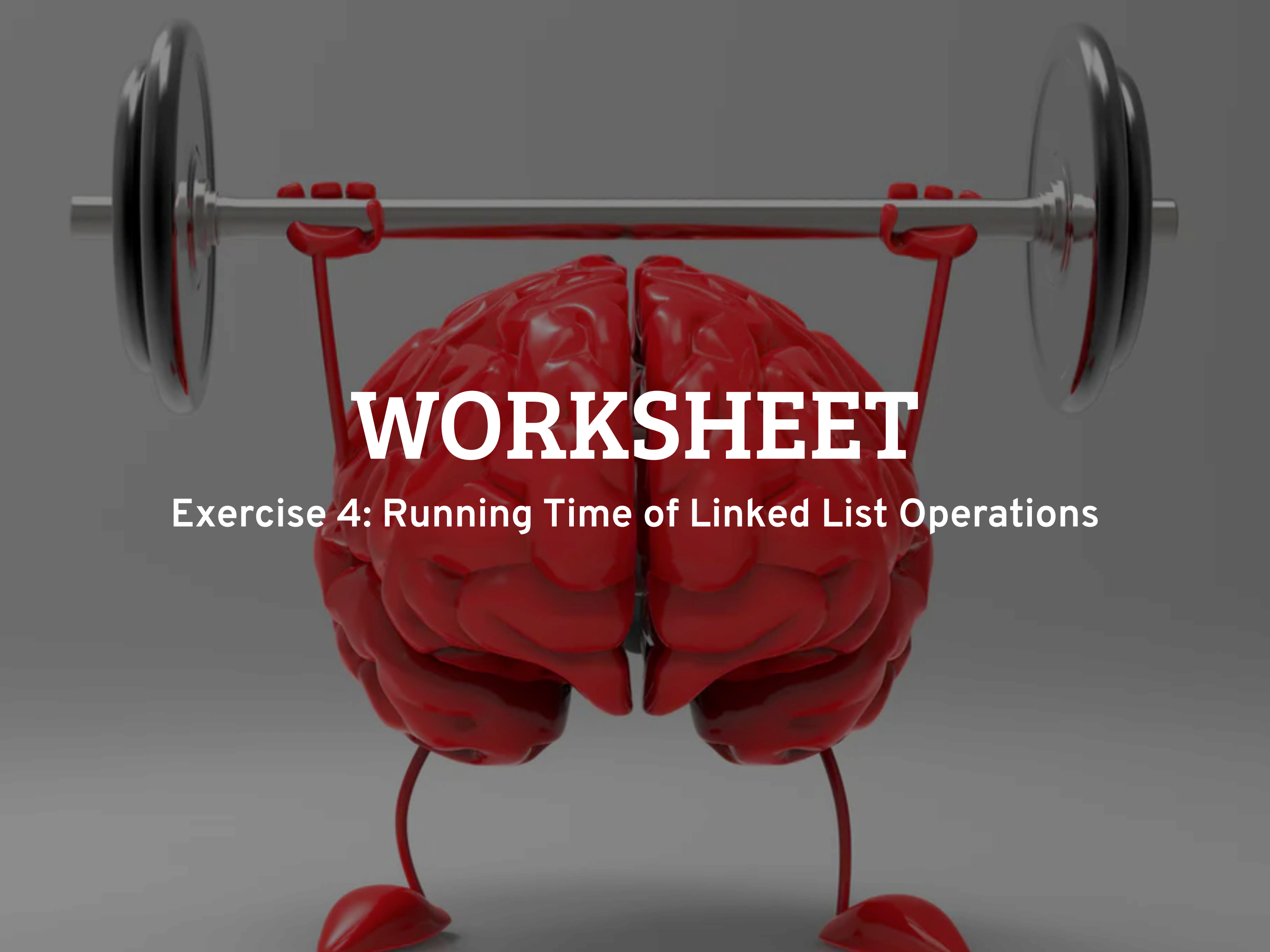
At which end of a Python list would it be best to insert an element at? Which index would it be easiest to remove?

At which end of a linked list would it be best to insert an element at? Which index would it be easiest to remove?

LinkedList size

We care about running time as a function of input size (length of the list).

For almost all linked list operations, running time is determined by the number of nodes traversed.



WORKSHEET

Exercise 4: Running Time of Linked List Operations

LinkedList.insert

```
def insert(self, i: int, item: Any) -> None:
    """..."""
    new_node = _Node(item)

    if i == 0:
        self._first, new_node.next = new_node, self._first
    else:
        curr = self._first
        curr_index = 0

        while not (curr is None or curr_index == i - 1):
            curr = curr.next
            curr_index = curr_index + 1

        if curr is None:
            raise IndexError
        else:
            curr.next, new_node.next = new_node, curr.next
```

Case 1: $i = 0$

Case 2: $i > 0$

*(more formal analysis
in the course notes)*

LinkedList.insert

```
def insert(self, i: int, item: Any) -> None:
    """..."""
    new_node = _Node(item)

    if i == 0:
        self._first, new_node.next = new_node, self._first
    else:
        curr = self._first
        curr_index = 0

        while not (curr is None or curr_index == i - 1):
            curr = curr.next
            curr_index = curr_index + 1

        if curr is None:
            raise IndexError
        else:
            curr.next, new_node.next = new_node, curr.next
```

How many iterations does the while loop take?

It iterates until...

Reaches the end of the list (i.e., n), OR
Reaches the correct index (i.e., i - 1)

Compare and contrast...

Operation (assuming $0 \leq i < n$)	Running time (list)	Running time (LinkedList)
Indexing (lst [<i>i</i>])	$\Theta(1)$	$\Theta(i)$
Insert into index <i>i</i>	$\Theta(n - i)$	$\Theta(i)$
Remove item at index <i>i</i>	$\Theta(n - i)$	$\Theta(i)$

For LinkedList, we assume that $i < n$, which simplifies our running times

See course notes for more details

Things to think about...

Consider storing more info: `_first`, `_last`, `_size`

How would these affect the implementation?

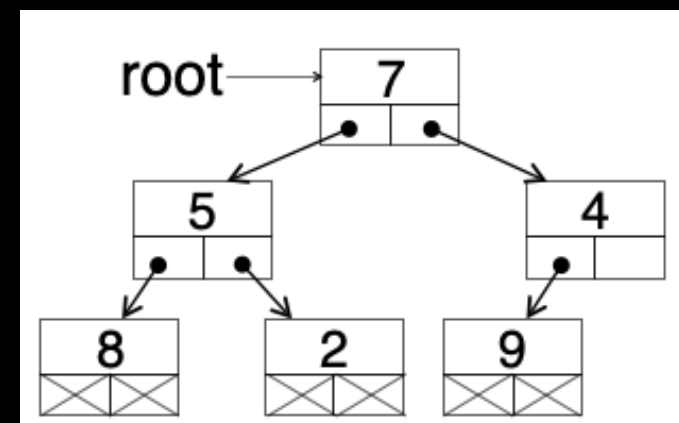
Why would we want to do this? Efficiency?

How else can we organize nodes?

Doubly-linked lists? (next + prev links for each node)

Circular linked lists? (last points back to first)

Hierarchical structure of nodes?



what's next?

Readings

Review this week's lectures:

Chapter 13

Next Week:

Recursion! Chapter 14

Important!

Project 1

Project 1 handout is out!

More info [here](#)